AI ASSISTED CODING

ASSIGNMENT-13.4

HT NO: 2303A510A2

NAME: M BHARATH

BATCH: 14

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Task 1: Refactoring Data Transformation Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

• Review the legacy code that computes transformed values using

an explicit loop.

• Use an AI tool to suggest a more Pythonic refactoring approach.

• Refactor the code using list comprehensions or helper functions

while preserving the output.

Legacy Code

values = [2, 4, 6, 8, 10]

doubled = []

for v in values:

doubled.append(v * 2)

print(doubled)

Expected Output

[4, 8, 12, 16, 20]

LEGACY CODE :

```
lab(13.4) > t1.py > ...
1   values = [2, 4, 6, 8, 10]
2   doubled = []
3   for v in values:
4   doubled.append(v * 2)
5   print(doubled)
6
```

OUTPUT:

```
PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/Python/Python313/py
13.4)/t1.py"
  File "c:\Users\akhil\OneDrive\Documents\AI(vs)\lab(13.4)\t1.py", line 4
    doubled.append(v * 2)
    ^^^^^^^
IndentationError: expected an indented block after 'for' statement on line 3
PS C:\Users\akhil\OneDrive\Documents\AI(vs)>
```

Explanation

1. values contains numbers.
2. An empty list doubled is created.
3. A for loop iterates through each value.
4. Each number is multiplied by 2.
5. The result is added to doubled using append().

PROMPT:

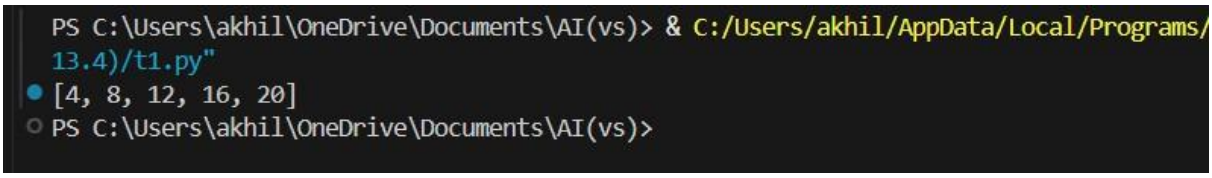
#Refactor the following legacy Python code to make it more Pythonic and readable

#using list comprehensions while keeping the output the same.

CODE :

```
7   #Refactor the following legacy Python code to make it more Pythonic and readable
8   #using list comprehensions while keeping the output the same.
9   values = [2, 4, 6, 8, 10]
10  doubled = [v * 2 for v in values]
11  print(doubled)
```

OUTPUT:



JUSTIFICATION:

The legacy code uses a manual loop and append() to double each value. This approach is verbose and less readable. Refactoring with list comprehension makes the code shorter, more Pythonic, and easier to maintain while producing the same output. List comprehensions are optimized in Python and improve both readability and performance.

Task 2: Improving Text Processing Code Readability

Scenario

You are working on a log message generator that builds messages word

by word.

Task Description

• Analyze the legacy code that constructs a sentence using repeated

string concatenation.

• Ask AI to suggest a more efficient and readable approach.

• Refactor the code accordingly while keeping the final output

unchanged.

Legacy Code

words = ["Refactoring", "with", "AI", "improves",

"quality"]

message = ""

for w in words:

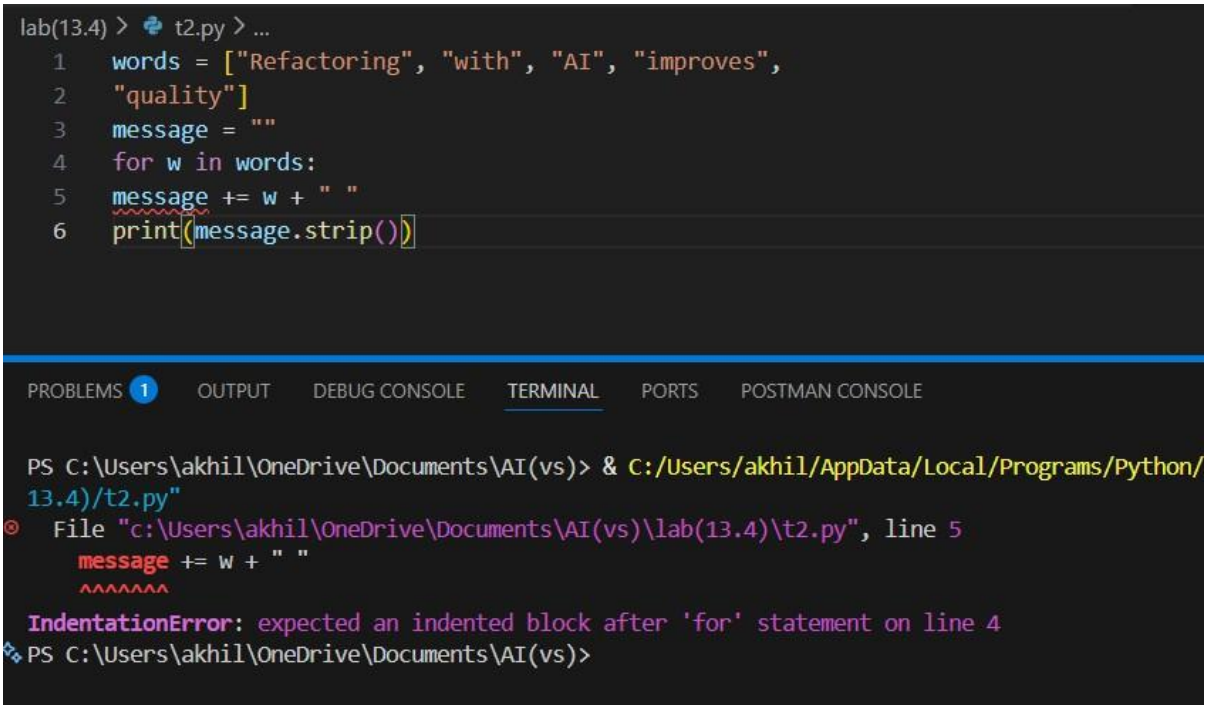
message += w + " "

print(message.strip())

Expected Output

Refactoring with AI improves quality

LEGACY CODE WITH OUTPUT:



Explanation

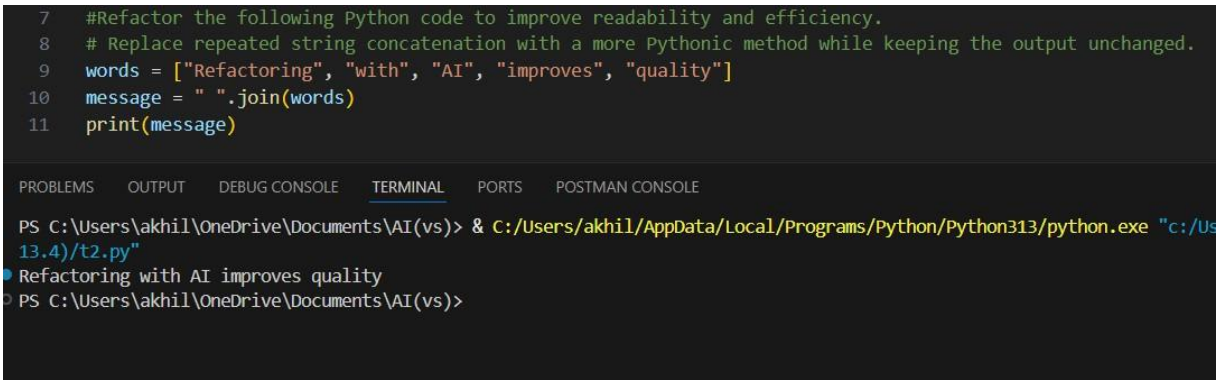
1. A list words contains several words.
2. message is initialized as an empty string.
3. A for loop iterates through each word in the list.
4. Each word is concatenated to message using +=.
5. A space " " is added after each word.
6. strip() removes the extra space at the end.
7. The final message is printed.

PROMPT:

#Refactor the following Python code to improve readability and efficiency.

Replace repeated string concatenation with a more Pythonic method while keeping the output unchanged.

CODE WITH OUTPUT:



JUSTIFICATION:

The legacy code repeatedly concatenates strings using +=, which is inefficient because Python strings are immutable and each concatenation creates a new string object. Using the join() method is more efficient and readable since it is specifically designed for combining multiple strings into one.

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may

or may not exist depending on deployment settings.

Task Description

• Review the legacy code that manually checks for dictionary keys.

• Use AI suggestions to refactor the code using safer dictionary

access methods.

• Ensure the behavior remains the same for missing keys.

Legacy Code

config = {"host": "localhost", "port": 8080}

if "timeout" in config:

print(config["timeout"])

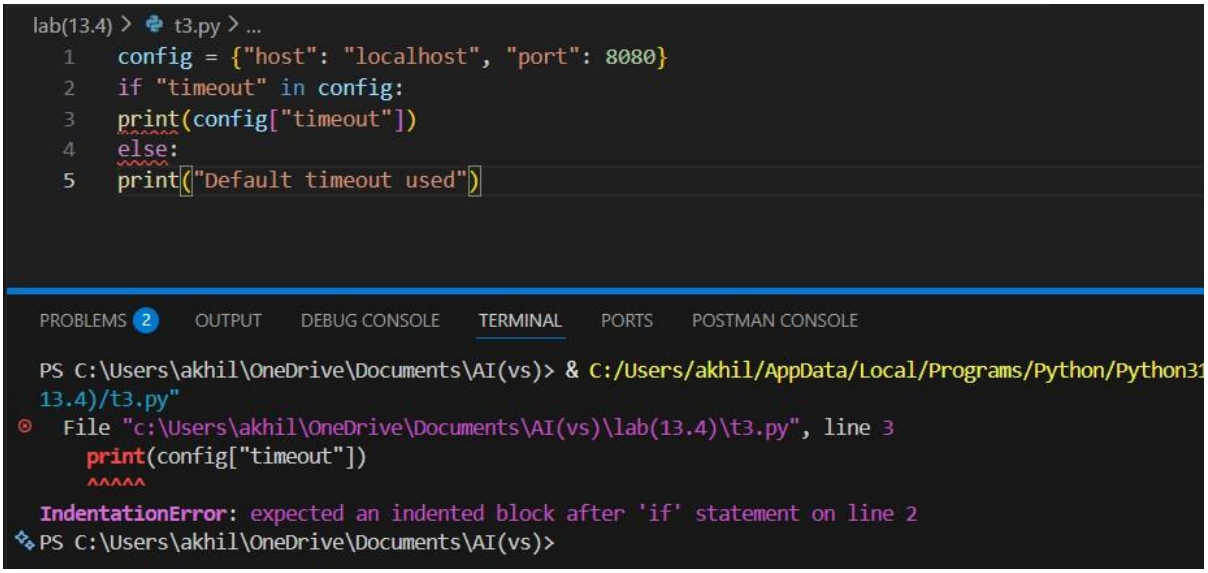
else:

print("Default timeout used")

Expected Output

Default timeout used

LEGACY CODE WITH OUTPUT:



Explanation

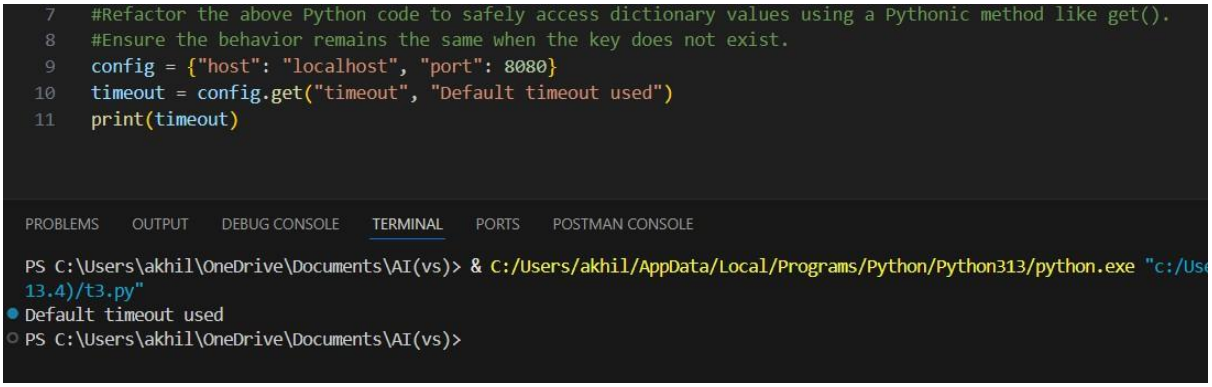
1. A dictionary config stores configuration settings.
2. It contains two keys:
    - "host"
    - "port"
3. The program checks whether "timeout" exists in the dictionary.
4. If it exists, its value is printed.
5. Otherwise, the message "Default timeout used" is printed.

PROMPT:

#Refactor the above Python code to safely access dictionary values using a Pythonic method like get().

#Ensure the behavior remains the same when the key does not exist.

CODE WITH OUTPUT:



JUSTIFICATION:

The legacy code manually checks if a dictionary key exists before accessing it. This increases code length and complexity. Using the dict.get() method provides safer and cleaner access to dictionary values, preventing potential errors and simplifying the code while preserving the same behavior for missing keys.

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations

based on user input.

Task Description

• Examine the multiple if–elif conditions used to determine

operations.

• Ask AI to suggest a cleaner, scalable alternative.

• Refactor the logic using mapping techniques while preserving

functionality.

Legacy Code

action = "divide"

x, y = 10, 2

if action == "add":

result = x + y

elif action == "subtract":

result = x - y

elif action == "multiply":

result = x * y

elif action == "divide":

result = x / y

else:

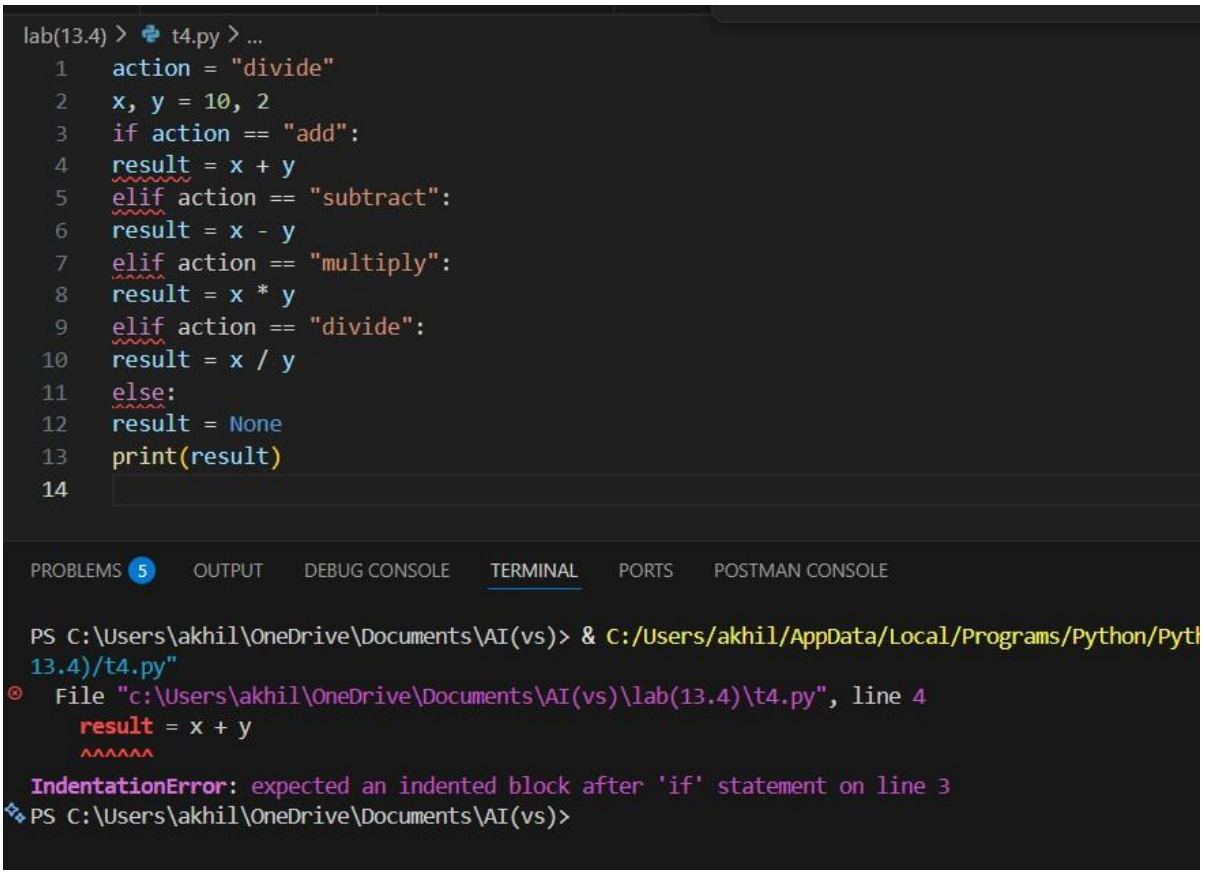
result = None

print(result)

Expected Output

5.0

LEGACY CODE WITH OUTPUT:



Explanation

1. action specifies the operation.
2. x and y are input numbers.
3. The program checks which operation to perform.
4. Each condition performs a different mathematical operation.
5. The result is printed.

PROMPT:

#Refactor the above Python code to replace multiple if–elif conditions with a cleaner and #more scalable approach using a mapping technique (such as a dictionary of functions). #Ensure the functionality and output remain the same.

CODE WITH OUTPUT:

```
#Refactor the above Python code to replace multiple if-elif conditions with a cleaner and
#more scalable approach using a mapping technique (such as a dictionary of functions).
#Ensure the functionality and output remain the same.
def add(x, y):
    return x + y
def subtract(x, y):
    return x - y
def multiply(x, y):
    return x * y
def divide(x, y):
    return x / y

action = "divide"
x, y = 10, 2
operations = {
    "add": add,
    "subtract": subtract,
    "multiply": multiply,
    "divide": divide
}
result = operations.get(action, lambda x, y: None)(x, y)
print(result)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/Python/Python313/python.
13.4)/t4.py"
5.0
PS C:\Users\akhil\OneDrive\Documents\AI(vs)>
```

JUSTIFICATION:

The legacy code relies on multiple if–elif conditions to determine operations. This approach becomes difficult to maintain as more operations are added. Using a dictionary mapping technique makes the code more scalable, cleaner, and easier to extend by simply adding new entries to the mapping**.**

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops

to find elements.

Task Description

• Identify the explicit loop used for searching an item.

• Use AI assistance to refactor the logic into a more concise and

readable form.

• Maintain the same output behavior.

Legacy Code

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
if item == "eraser":
found = True
break
print("Item Available" if found else "Item Not Available")
```

Expected Output

Item Available

LEGACY CODE WITH OUTPUT:

Explanation

1. A list inventory contains store items.
2. found is initialized as False.
3. The program loops through each item.
4. If "eraser" is found:
    - found becomes True
    - the loop stops using break
5. A message is printed depending on whether the item was found.

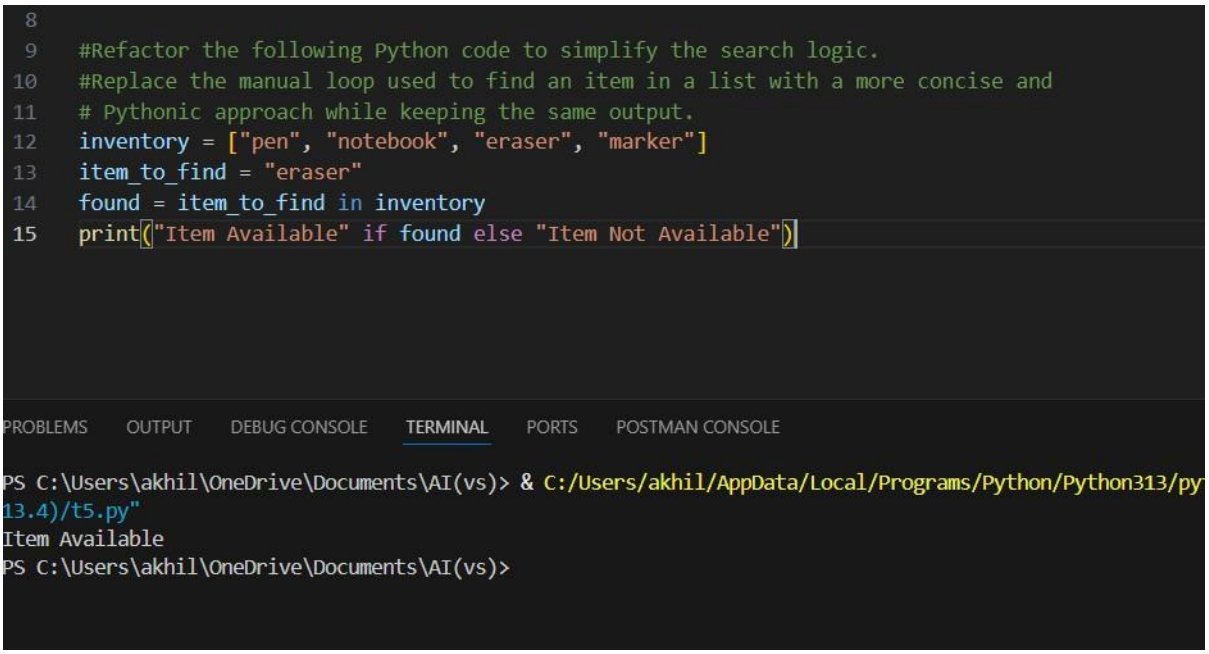
PROMPT:

#Refactor the following Python code to simplify the search logic.

#Replace the manual loop used to find an item in a list with a more concise and

Pythonic approach while keeping the same output.

CODE WITH OUTPUT:



JUSTIFICATION:

The legacy code uses a manual loop and a flag variable to search for an item in a list. Python provides a built-in membership operator (in) that performs this check directly. Using in simplifies the code, improves readability, and eliminates unnecessary variables while maintaining the same functionality.